Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Математические основы интеллектуальных систем»

За 3 семестр

Тема: «Определение компонент сильной связности в орграфе. Определение компонент двусвязности для неориентированного графа»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Игнатюк Д.А.

Проверил:
Козинский А.А.

Брест, 2026

Лабораторная работа №5

«Определение компонент сильной связности в орграфе. Определение компонент двусвязности для неориентированного графа»

Цель работы: освоить алгоритмы нахождения компонент сильной связности ориентированного графа и компонент двусвязности (точек сочленения и мостов) неориентированного графа и реализовать их на языке С.

Ход работы

Код

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
// ПРЕДСТАВЛЕНИЕ ГРАФА: СПИСКИ СМЕЖНОСТИ
typedef struct Edge {
    int v;
    int id;             // номер ребра в файле (для неориентированного — один на пару)
    struct Edge* next;
} Edge;
typedef struct {
    int   n;
    int   m;
    int   directed;     // 0 — неориентированный, 1 — ориентированный
    Edge** adj;
} Graph;
// Добавление ребра/дуги
static void add_edge(Graph* g, int u, int v, int id)
{
    Edge* p = (Edge*)malloc(sizeof(Edge));
    p->v = v; p->id = id; p->next = g->adj[u]; g->adj[u] = p;
    if (!g->directed) {
        p = (Edge*)malloc(sizeof(Edge));
        p->v = u; p->id = id; p->next = g->adj[v]; g->adj[v] = p;
    }
}
// Чтение графа
static int read_graph(const char* fname, Graph* g)
{
    FILE* f = fopen(fname, "r");
    if (!f) { fprintf(stderr, "ne otkryt %s\n", fname); return 0; }
    char line[256];
    int directed = 0, n = 0;
    int header_ok = 0;
    // Первая непустая строка — флаг directed
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &directed) == 1) { header_ok = 1; break; }
    }
    if (!header_ok) { fprintf(stderr, "%s: net flaga\n", fname); fclose(f); return 0; }
    // Вторая непустая — n
    header_ok = 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &n) == 1) { header_ok = 1; break; }
    }
```

```
    if (!header_ok || n < 0) { fprintf(stderr, "%s: plokhoye n\n",
fname); fclose(f); return 0; }
    g->n = n;
    g->m = 0;
    g->directed = directed ? 1 : 0;
    g->adj = (Edge**)calloc(n + 1, sizeof(Edge*));
    if (!g->adj) { fclose(f); return 0; }
    int edge_id = 0;
    while (fgets(line, sizeof(line), f)) {
        int u = 0, v = 0;
        int got = sscanf(line, "%d %d", &u, &v);
        if (got < 2) continue;
        if (u < 1 || u > g->n || v < 1 || v > g->n) {
            fprintf(stderr, "%s: rebro (%d,%d) vne [1..%d], propusk\n",
                fname, u, v, g->n);
            continue;
        }
        if (u == v) {
            fprintf(stderr, "%s: petlya (%d,%d), propusk\n", fname, u,
v);
            continue;
        }
        edge_id++;
        add_edge(g, u, v, edge_id);
        g->m++;
    }
    fclose(f);
    return 1;
}
static void free_graph(Graph* g)
{
    for (int i = 1; i <= g->n; i++) {
        Edge* e = g->adj[i];
        while (e) { Edge* nx = e->next; free(e); e = nx; }
    }
    free(g->adj);
    g->adj = NULL;
    g->n = g->m = 0;
}
/*  ЗАДАЧА 1: КОМПОНЕНТЫ СИЛЬНОЙ СВЯЗНОСТИ (алгоритм Тарьяна)
 *  Итеративный, без рекурсии.
 *  tin[v]   — время входа в v
 *  low[v]   — минимальное tin, достижимое из v
 *  in_stack — v лежит в стеке
 *  stack[]  — стек вершин текущей компоненты */
typedef struct {
    int v;
    Edge* iter;   // указатель на текущую дугу в adj[v]
} DFSFrame;
static void tarjan_scc(const Graph* g, int* comp)
{
    int N = g->n;
    int* tin = (int*)calloc(N + 1, sizeof(int));
    int* low = (int*)calloc(N + 1, sizeof(int));
    int* in_stack = (int*)calloc(N + 1, sizeof(int));
    int* stack = (int*)malloc((N + 1) * sizeof(int));
    DFSFrame* frames = (DFSFrame*)malloc((N + 1) * sizeof(DFSFrame));
    int time = 0;
    int top = 0;
```

```
int comp_cnt = 0;
for (int i = 1; i <= N; i++) comp[i] = 0;
for (int start = 1; start <= N; start++) {
    if (tin[start]) continue;
    // Запуск DFS из start
    int fsp = 0;
    frames[fsp].v = start;
    frames[fsp].iter = g->adj[start];
    fsp++;
    tin[start] = low[start] = ++time;
    stack[top++] = start;
    in_stack[start] = 1;
    while (fsp > 0) {
        DFSFrame* fr = &frames[fsp - 1];
        int v = fr->v;
        Edge* e = fr->iter;
        if (e != NULL) {
            fr->iter = e->next;
            int u = e->v;
            if (!tin[u]) {
                // Идём в новую вершину
                tin[u] = low[u] = ++time;
                stack[top++] = u;
                in_stack[u] = 1;
                frames[fsp].v = u;
                frames[fsp].iter = g->adj[u];
                fsp++;
            }
            else if (in_stack[u]) {
                // Обратная дуга в стек — обновляем low
                if (tin[u] < low[v]) low[v] = tin[u];
            }
        }
        else {
            // Все дуги обработаны — снимаем кадр
            fsp--;
            if (fsp > 0) {
                int parent = frames[fsp - 1].v;
                if (low[v] < low[parent]) low[parent] = low[v];
            }
            // v — корень компоненты?
            if (tin[v] == low[v]) {
                comp_cnt++;
                while (top > 0) {
                    int w = stack[--top];
                    in_stack[w] = 0;
                    comp[w] = comp_cnt;
                    if (w == v) break;
                }
            }
        }
    }
}
printf("Komponent silnoy svyaznosti: %d\n", comp_cnt);
for (int c = 1; c <= comp_cnt; c++) {
    printf("  KSS %d: ", c);
    for (int i = 1; i <= N; i++) if (comp[i] == c) printf("%d ", i);
    printf("\n");
}
```

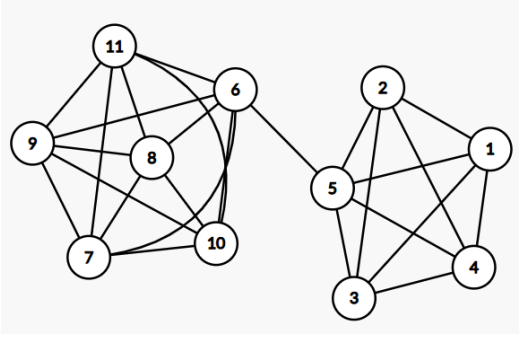
```
    free(tin); free(low); free(in_stack); free(stack); free(frames);
}
/*  ЗАДАЧА 2: ТОЧКИ СОЧЛЕНЕНИЯ, МОСТЫ, БЛОКИ
 *  Для НЕориентированного графа.
 *  tin/low как в Тарьяне, но нельзя возвращаться по тому же ребру —
храним ID ребра.
 *  Храним в кадре: v, iter (указатель в adj[v]), parent_edge_id
 *  Для определения моста: low[u] > tin[v] → мост (v,u)
 *  Точка сочленения: low[u] >= tin[v] (не корень), либо корень с 2+
детьми */
typedef struct {
    int v;
    Edge* iter;
    int parent_edge_id;
    int parent_v;
} DFSFrame2;
static void biconnected(const Graph* g)
{
    int N = g->n;
    int* tin = (int*)calloc(N + 1, sizeof(int));
    int* low = (int*)calloc(N + 1, sizeof(int));
    int* is_art = (int*)calloc(N + 1, sizeof(int));   // точка
сочленения
    int* is_bridge = (int*)calloc(g->m + 1, sizeof(int));  // мост по id
    DFSFrame2* frames = (DFSFrame2*)malloc((N + 1) * sizeof(DFSFrame2));
    int* child_count = (int*)calloc(N + 1, sizeof(int));
    int time = 0;
    for (int start = 1; start <= N; start++) {
        if (tin[start]) continue;
        int fsp = 0;
        frames[fsp].v = start;
        frames[fsp].iter = g->adj[start];
        frames[fsp].parent_edge_id = -1;
        frames[fsp].parent_v = 0;
        fsp++;
        tin[start] = low[start] = ++time;
        while (fsp > 0) {
            DFSFrame2* fr = &frames[fsp - 1];
            int v = fr->v;
            Edge* e = fr->iter;
            if (e != NULL) {
                fr->iter = e->next;
                int u = e->v;
                // Не возвращаемся по тому же ребру
                if (e->id == fr->parent_edge_id) continue;
                if (!tin[u]) {
                    // Ребёнок
                    child_count[v]++;
                    tin[u] = low[u] = ++time;
                    frames[fsp].v = u;
                    frames[fsp].iter = g->adj[u];
                    frames[fsp].parent_edge_id = e->id;
                    frames[fsp].parent_v = v;
                    fsp++;
                }
                else {
                    // Обратное ребро (в предка)
                    if (tin[u] < low[v]) low[v] = tin[u];
                }
```

```
            }
            else {
                // Снимаем кадр
                fsp--;
                if (fsp > 0) {
                    DFSFrame2* parent = &frames[fsp - 1];
                    int pv = parent->v;
                    int peid = frames[fsp].parent_edge_id;
                    // low[pv] = min(low[pv], low[v])
                    if (low[v] < low[pv]) low[pv] = low[v];
                    // Мост?
                    if (low[v] > tin[pv]) {
                        is_bridge[peid] = 1;
                    }
                    // Точка сочленения (не корень)?
                    if (low[v] >= tin[pv] && pv != start) {
                        is_art[pv] = 1;
                    }
                    else if (low[v] >= tin[pv] && pv == start &&
child_count[start] > 1) {
                        is_art[pv] = 1;
                    }
                }
            }
        }
    }
    // Вывод
    printf("\nTochki sochleneniya:\n  ");
    int cnt_art = 0;
    for (int i = 1; i <= N; i++) if (is_art[i]) { printf("%d ", i);
cnt_art++; }
    if (cnt_art == 0) printf("net");
    printf("\n");
    printf("Mosty:\n  ");
    int cnt_bridge = 0;
    for (int u = 1; u <= N; u++) {
        for (const Edge* e = g->adj[u]; e; e = e->next) {
            if (e->id == 0) continue;
            if (is_bridge[e->id] && u < e->v) {
                printf("%d-%d ", u, e->v);
                cnt_bridge++;
            }
        }
    }
    if (cnt_bridge == 0) printf("net");
    printf("\n");
    free(tin); free(low); free(is_art); free(is_bridge);
    free(frames); free(child_count);
}
int main(int argc, char** argv)
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <graph-file>\n", argv[0]);
        return 1;
    }
    Graph g;
    if (!read_graph(argv[1], &g)) return 1;
    printf("Fayl: %s\n", argv[1]);
```

```
    printf("Tip: %s\n", g.directed ? "orientirovannyy" :
"neorientirovannyy");
    printf("Vershin: %d, reber/dug: %d\n\n", g.n, g.m);
    if (g.directed) {
        //КСС
        printf("=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===\n");
        int* comp = (int*)malloc((g.n + 1) * sizeof(int));
        tarjan_scc(&g, comp);
        free(comp);
    }
    else {
        // Двусвязность
        printf("=== TOCHKI SOCHLENENIYA I MOSTY ===\n");
        biconnected(&g);
    }
    free_graph(&g);
    return 0;
}
```

Проверки:

Тест001 - два полных K_5, соединённых одним мостом

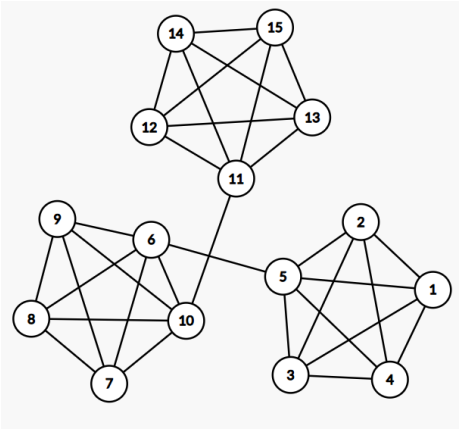



```
Fayl: tests/test001.txt
Tip: neorientirovannyy
Vershin: 11, reber/dug: 26

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  5 6
Mosty:
  5-6
```

Тест002 - 3 K_5, соединённые двумя мостами

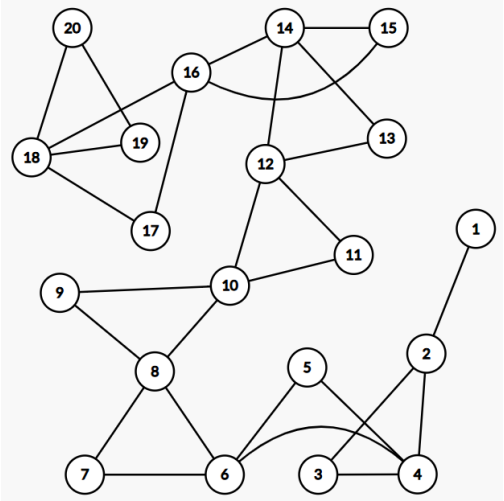



```
Fayl: tests/test002.txt
Tip: neorientirovannyy
Vershin: 16, reber/dug: 32

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  5 6 10 11
Mosty:
  5-6 10-11
```

Тест003 - Цепь из 20 с добавлением «лестницы» (25 вершин)

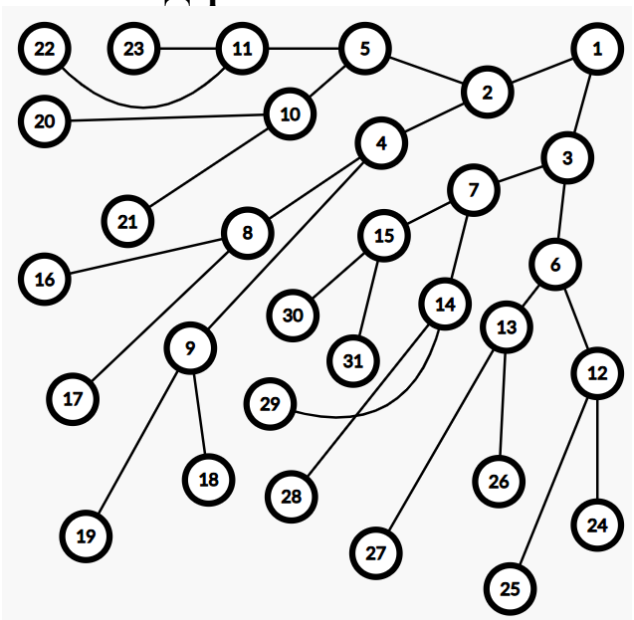



```
Fayl: tests/test003.txt
Tip: neorientirovannyy
Vershin: 20, reber/dug: 28

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  2 4 6 8 10 12 14 16 18
Mosty:
  1-2
```

Тест004 - Дерево-«ёлочка» на 31

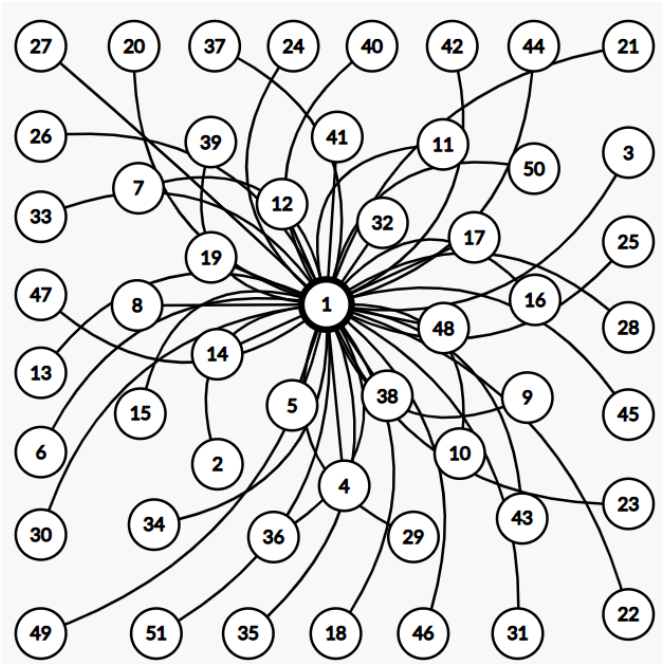



```
Fayl: tests/test004.txt
Tip: neorientirovannyy
Vershin: 31, reber/dug: 30

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Mosty:
  1-3 1-2 2-5 2-4 3-7 3-6 4-9 4-8 5-11 5-10 6-13 6-
12 7-15 7-14 8-17 8-16 9-19 9-18 10-21 10-20 11-23
11-22 12-25 12-24 13-27 13-26 14-29 14-28 15-31 15-
30
```

Тест005 - Звезда с 50 листьями (51 вершина)




```
Fayl: tests/test005.txt
Tip: neorientirovannyy
Vershin: 51, reber/dug: 50

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  1
Mosty:
  1-51 1-50 1-49 1-48 1-47 1-46 1-45 1-44 1-43 1-42 1-
41 1-40 1-39 1-38 1-37 1-36 1-35 1-34 1-33 1-32 1-31 1
-30 1-29 1-28 1-27 1-26 1-25 1-24 1-23 1-22 1-21 1-20
1-19 1-18 1-17 1-16 1-15 1-14 1-13 1-12 1-11 1-10 1-9
1-8 1-7 1-6 1-5 1-4 1-3 1-2
```

Тест006 - «Гусеница» на 100

```
Fayl: tests/test006.txt
Tip: neorientirovannyy
Vershin: 100, reber/dug: 99

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
Mosty:
  1-51 1-2 2-52 2-3 3-53 3-4 4-54 4-5 5-55 5-6 6-56 6-7 7-57 7-8 8-58 8-9 9-59 9-10 10-60 10-11 11-61 11-12 12-62 12-13 13-63 13-14 14-64 14-15 15-65 15-16
16-66 16-17 17-67 17-18 18-68 18-19 19-69 19-20 20-70 20-21 21-71 21-22 22-72 22-23 23-73 23-24 24-74 24-25 25-75 25-26 26-76 26-27 27-77 27-28 28-78 28-29
29-79 29-30 30-80 30-31 31-81 31-32 32-82 32-33 33-83 33-34 34-84 34-35 35-85 35-36 36-86 36-37 37-87 37-38 38-88 38-39 39-89 39-40 40-90 40-41 41-91 41-42
42-92 42-43 43-93 43-44 44-94 44-45 45-95 45-46 46-96 46-47 47-97 47-48 48-98 48-49 49-99 49-50 50-100
```

Тест007 – 101 вершина

```
Fayl: tests/test007.txt
Tip: neorientirovannyy
Vershin: 101, reber/dug: 160

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  5 6 10 11 15 16 20 21 25 26 30 31 35 36 40 41 45 46 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84
85 86 87 88 89 90 91 92 93 94 95
Mosty:
  5-51 6-55 10-56 11-60 15-61 16-65 20-66 21-70 25-71 26-75 30-76 31-80 35-81 36-85 40-86 41-90 45-91 46-95 51-96 51-52 52-97 52-53 53-98 53-54 54-99 54-55
55-100 56-101 56-57 57-58 58-59 59-60 61-62 62-63 63-64 64-65 66-67 67-68 68-69 69-70 71-72 72-73 73-74 74-75 76-77 77-78 78-79 79-80 81-82 82-83 83-84 84-8
5 86-87 87-88 88-89 89-90 91-92 92-93 93-94 94-95
```

Тест008 – «двойная гусеница» на 200 вершин

```
Fayl: tests/test008.txt
Tip: neorientirovannyy
Vershin: 200, reber/dug: 208

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  91 92 93 94 95 96 97 98 99 191 192 193 194 195 196 197 198 199
Mosty:
  91-92 92-93 93-94 94-95 95-96 96-97 97-98 98-99 99-100 191-192 192-193 193-194 194-195 195-196 196-197 197-198 198-199 199-200
```

Тест009 – Цепь из 500 с 250 листьями (750 вершин)

```
Fayl: tests/test009.txt
Tip: neorientirovannyy
Vershin: 750, reber/dug: 749

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
56 57 58 59 60 61 62 63 64 65 66 67 68
```

```
457 458 459 460 461 462 463 464 465
496 497 498 499 500
Mosty:
  1-2 2-501 2-3 3-4 4-502 4-5 5-6 6
0 20-510 20-21 21-22 22-511 22-23 2
-518 36-37 37-38 38-519 38-39 39-40
 52-53 53-54 54-527 54-55 55-56 56-
```

```
65-466 466-733 466-467 467-468 468-734 468-469 469-470 470-735 470-471 471-472 472-736 472-473 473-474 474-737 474
39 478-479 479-480 480-740 480-481 481-482 482-741 482-483 483-484 484-742 484-485 485-486 486-743 486-487 487-488
91-492 492-746 492-493 493-494 494-747 494-495 495-496 496-748 496-497 497-498 498-749 498-499 499-500 500-750
```

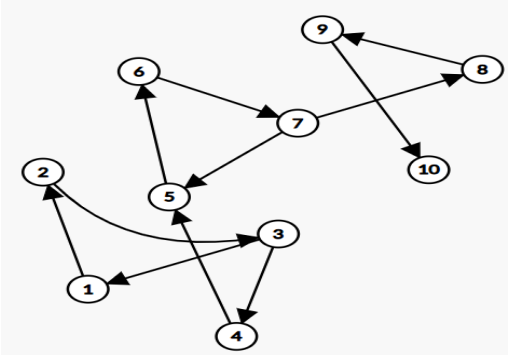
Тест010 – K_{1000} + мост + K_{1000} (2001 вершина)

```
Fayl: tests/test010.txt
Tip: neorientirovannyy
Vershin: 2001, reber/dug: 999002

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  1000 1001 2001
Mosty:
  1000-2001 1001-2001
```

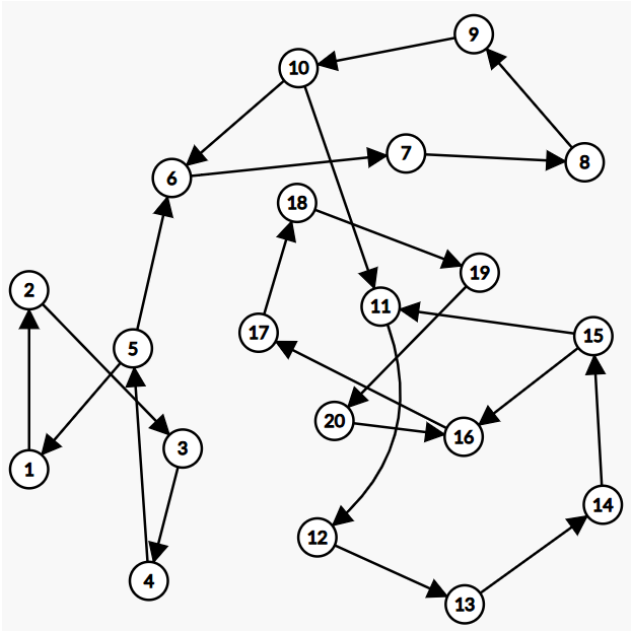
Тест101 – Цепь + хвосты (10 вершин)




```
Fayl: tests/test101.txt
Tip: orientirovannyy
Vershin: 10, reber/dug: 11

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 6
  KSS 1: 10
  KSS 2: 9
  KSS 3: 8
  KSS 4: 5 6 7
  KSS 5: 4
  KSS 6: 1 2 3
```

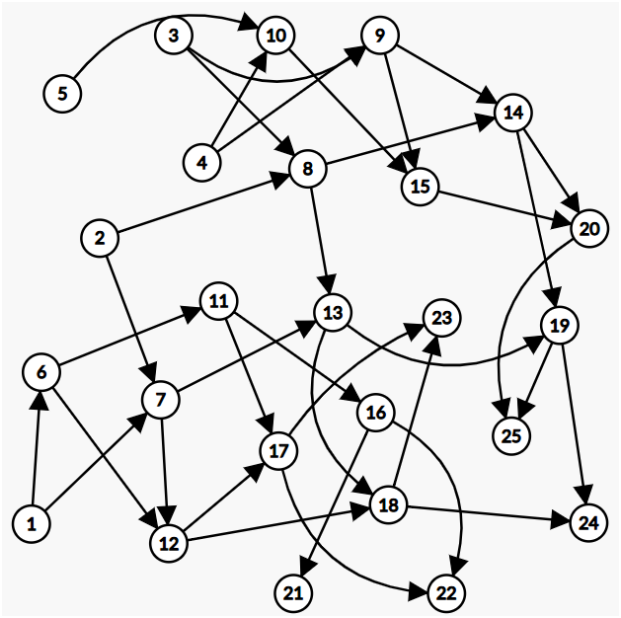
Тест102 – «Слоёный» граф (20 вершин)




```
Fayl: tests/test102.txt
Tip: orientirovannyy
Vershin: 20, reber/dug: 23

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 4
  KSS 1: 16 17 18 19 20
  KSS 2: 11 12 13 14 15
  KSS 3: 6 7 8 9 10
  KSS 4: 1 2 3 4 5
```

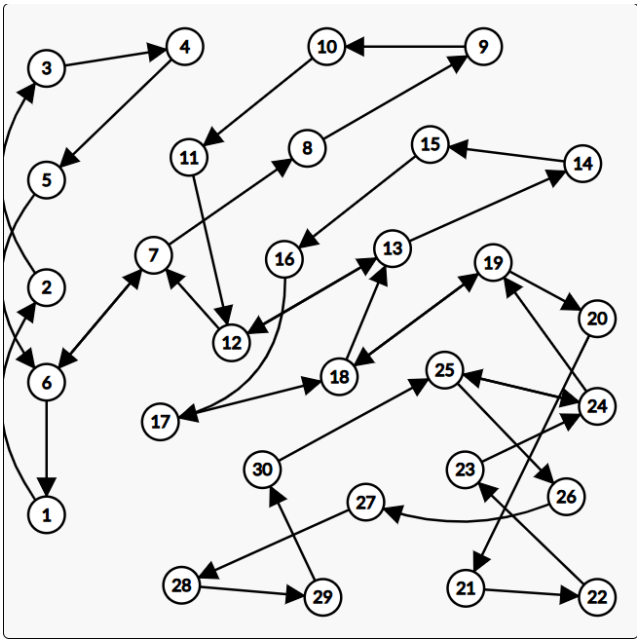
Тест103 – DAG с «ромбами» (25 вершин)




```
Fayl: tests/test103.txt
Tip: orientirovannyy
Vershin: 25, reber/dug: 36

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 25
  KSS 1: 25
  KSS 2: 24
```

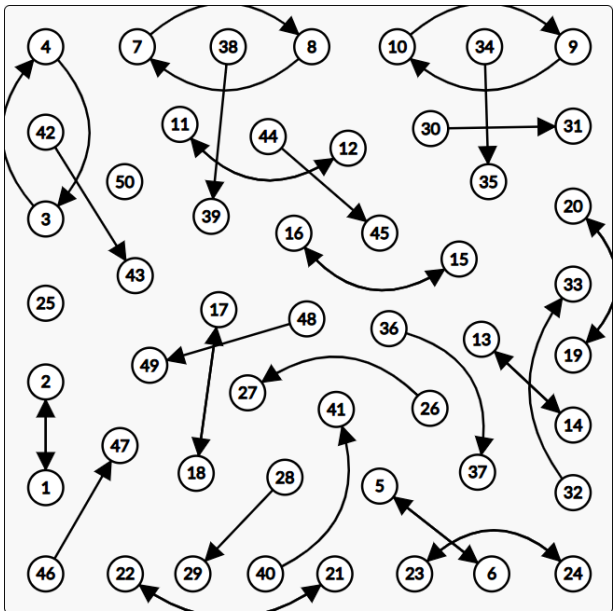
Тест104 – 5 циклов с обратными дугами (30 вершин)



```
Fayl: tests/test104.txt
Tip: orientirovannyy
Vershin: 30, reber/dug: 38

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 1
  KSS 1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
```

Test105 – «Два острова» (50 вершин)




```
Fayl: tests/test105.txt
Tip: orientirovannyy
Vershin: 50, reber/dug: 36

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 38
  KSS 1: 1 2
  KSS 2: 3 4
  KSS 3: 5 6
  KSS 4: 7 8
  KSS 5: 9 10
  KSS 6: 11 12
  KSS 7: 13 14
  KSS 8: 15 16
```

Test106 – Цикл C_{100} (100 вершин)

```
Fayl: tests/test106.txt
Tip: orientirovannyy
Vershin: 100, reber/dug: 100

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 1
  KSS 1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62
63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93
94 95 96 97 98 99 100
```

Test107 – DAG-цепь (200 вершин)

```
Fayl: tests/test107.txt
Tip: orientirovannyy
Vershin: 200, reber/dug: 199

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 200
  KSS 1: 200
  KSS 2: 199
  KSS 3: 198
  KSS 4: 197
```

```
KSS 191: 10
KSS 192: 9
KSS 193: 8
KSS 194: 7
KSS 195: 6
KSS 196: 5
KSS 197: 4
KSS 198: 3
KSS 199: 2
KSS 200: 1
```

Test108 – Случайный DAG (500 вершин)

```
Fayl: tests/test108.txt
Tip: orientirovannyy
Vershin: 500, reber/dug: 1230

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 500
  KSS 1: 483
  KSS 2: 429
```

Тест109 – Слоёный из 10 циклов (1000 вершин)

```
Fayl: tests/test109.txt
Tip: orientirovannyy
Vershin: 1000, reber/dug: 1009

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 10
  KSS 1: 901 902 903 904 905 906 907 908 909 910
 934 935 936 937 938 939 940 941 942 943 944 945
 969 970 971 972 973 974 975 976 977 978 979 980
  KSS 2: 801 802 803 804 805 806 807 808 809 810
 834 835 836 837 838 839 840 841 842 843 844 845
```

Тест110 – Цикл C_{1000} (1000 вершин)

```
Fayl: tests/test110.txt
Tip: orientirovannyy
Vershin: 1000, reber/dug: 1000

=== KOMPONENTY SILNOY SVYAZNOSTI (Tarjan) ===
Komponent silnoy svyaznosti: 1
  KSS 1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 1
 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62
4 95 96 97 98 99 100 101 102 103 104 105 106 1
```

Тест200 – Полное бинарное дерево, 1 048 575 вершин (~42с)

```
Fayl: tests/test200.txt
Tip: neorientirovannyy
Vershin: 1048575, reber/dug: 1048574

=== TOCHKI SOCHLENENIYA I MOSTY ===

Tochki sochleneniya:
  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 1
50 51 52 53 54 55 56 57 58 59 60 61 62 63
 97 98 99 100 101 102 103 104 105 106 107
2 133 134 135 136 137 138 139 140 141 142
```

Вывод: реализована программа на C для поиска компонент сильной связности ориентированного графа и компонент двусвязности неориентированного графа. Программа протестирована на 21 тесте: 10 неориентированных графов (до 2001 вершины), 10 ориентированных (до 1000 вершин) и 1 стресс-тест на 1 048 575 вершин. Во всех случаях результаты совпали с теоретическими ожиданиями.